ObsPair modelling – Neural Network variant

Andrew Hoskins

March 2026

1 Background

This document describes a neural network implementation of the CLESSO (Community-Level Species Similarity Observations) model. The core ecological framework – estimating species richness (α) and compositional turnover (β) from observation-pair data – is identical to the TMB version described in the companion document. The key difference is that the parametric sub-models (linear terms, P-splines, I-splines) are replaced with neural networks, while preserving the ecological constraints that make the model interpretable.

The motivation for this variant is computational. The TMB implementation relies on a Laplace approximation over site-level random effects (u_i), which requires repeated sparse Cholesky factorisations of the joint precision matrix. For large numbers of sites ($> 10^4$), this becomes prohibitively expensive – often requiring days to converge. A neural network trained with mini-batch stochastic gradient descent eliminates the Laplace approximation entirely, reducing training from days to minutes while retaining (and potentially improving upon) the flexibility of the spline-based approach.

2 Pair constructs and likelihood

The data pipeline is shared with the TMB variant. We sample a set of observation pairs $P = P_{\text{within}} \cup P_{\text{between}}$ from species occurrence records. For each pair p we record:

- i_p, j_p: the sites involved
- $y_p = 0$ if the species identities match, $y_p = 1$ otherwise
- w_p: a weight that balances the four cells {within-match, within-mismatch, between-match, between-mismatch}

Under the equal-draw assumption from Hoskins et al. (2020), the match probability decomposes as:

$$p_p = \begin{cases} \dfrac{1}{\alpha_i} & \text{if } i_p = j_p \text{ (within-site pair)} \\ S_{i,j} \cdot \dfrac{\alpha_i + \alpha_j}{2\,\alpha_i\,\alpha_j} & \text{if } i_p \neq j_p \text{ (between-site pair)} \end{cases} \tag{1}$$

where $\alpha_i > 1$ is the species richness at site i and $S_{i,j} = \exp(-\eta_{i,j}) \in [0, 1]$ is the Sørensen-type similarity derived from turnover $\eta_{i,j} \geq 0$.

The loss function is the weighted negative log-likelihood of the Bernoulli model plus a regularisation penalty:

$$\mathcal{L} = -\frac{1}{\sum_p w_p} \sum_{p \in P} w_p \Big[(1 - y_p) \log p_p + y_p \log(1 - p_p) \Big] + \lambda_{\text{lb}} \cdot \mathcal{R}_\alpha \tag{2}$$

This is identical to the TMB formulation. The difference lies entirely in *how α_i and $\eta_{i,j}$ are parameterised.*

3 Alpha sub-model: richness network

In the TMB model, richness is parameterised as:

$$\log(\alpha_i - 1) = \theta_0 + \mathbf{z}_i^\top \boldsymbol{\theta} + \sum_k g_k(z_{k,i}^{\text{raw}}) + u_i$$

with linear terms, P-spline smooths $g_k(\cdot)$, and site-level random effects $u_i \sim \mathcal{N}(0, \sigma_u^2)$.

In the neural network variant, all of these components are replaced by a single multi-layer perceptron (MLP):

$$\alpha_i = \text{softplus}(f_\alpha(\mathbf{z}_i; \boldsymbol{W}_\alpha)) + 1 \tag{3}$$

where f_α is a feedforward network with L hidden layers:

$$f_\alpha(\mathbf{z}) = \mathbf{W}^{(L+1)}\, \sigma\Big(\mathbf{W}^{(L)}\, \sigma\Big(\cdots \sigma\Big(\mathbf{W}^{(1)}\, \mathbf{z} + \mathbf{b}^{(1)} \Big) \cdots \Big) + \mathbf{b}^{(L)} \Big) + b^{(L+1)} \tag{4}$$

Here $\sigma(\cdot)$ is a nonlinear activation, and $\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}$ are the weights and biases at layer ℓ. The default architecture uses four hidden layers with dimensions $[256, 128, 64, 32]$ with SiLU (Sigmoid Linear Unit, $x \cdot \sigma(x)$) activation and dropout regularisation between layers. SiLU was chosen over ReLU for its smooth gradient landscape, which improves optimisation stability.

The output activation $\text{softplus}(\cdot) + 1$ guarantees $\alpha_i > 1$, matching the TMB constraint. The output bias is initialised so that $\text{softplus}(1.1) + 1 \approx 2.7$, providing a reasonable starting point for species richness. The key advantage is that the network simultaneously learns:

- the linear effects captured by $\mathbf{z}^\top \boldsymbol{\theta}$ in the TMB model

- the nonlinear smooth terms $g_k(\cdot)$ captured by P-splines
- the site-level heterogeneity captured by random effects u_i

all as a single differentiable function, without requiring the Laplace approximation needed by the random effects.

Note on random effects. The flexibility of the hidden layers largely subsumes what u_i captured in the TMB model. The P-spline basis with K knots per covariate provides $\mathcal{O}(K)$ degrees of freedom; the network provides $\mathcal{O}(\sum_\ell d_\ell \cdot d_{\ell+1})$ parameters, which is typically much larger. If additional site-level variation is needed, a learned per-site embedding could be added, though this risks overfitting.

3.1 Fourier positional encoding

Raw longitude and latitude coordinates are poor features for neural networks because they impose a linear geometry on what is a spherical surface. To provide the alpha network with multi-scale spatial information, we optionally replace (or supplement) raw coordinates with Fourier positional encodings:

$$\text{Fourier}_k(\text{lon}, \text{lat}) = \left[\sin\left(\frac{2\pi\,\text{lon}}{\lambda_k}\right),\ \cos\left(\frac{2\pi\,\text{lon}}{\lambda_k}\right),\ \sin\left(\frac{2\pi\,\text{lat}}{\lambda_k}\right),\ \cos\left(\frac{2\pi\,\text{lat}}{\lambda_k}\right)\right] \tag{5}$$

with wavelengths $\lambda_k = \lambda_{\max}/2^k$ for $k = 0, \ldots, N_f - 1$. With $\lambda_{\max} = 40°$ and $N_f = 5$, this produces wavelengths $[40, 20, 10, 5, 2.5]$ degrees, spanning continental to regional scales. Each frequency contributes 4 features (sin/cos for each coordinate), so $N_f = 5$ adds 20 features to $\mathbf{z}_i$.

This encoding allows the network to learn spatially structured richness patterns at multiple scales without requiring an explicit spatial random effect. When raw coordinates are excluded (`exclude_coords_from_alpha=True`), the Fourier features are the sole source of geographic information.

3.2 Alpha lower-bound penalty

As in the TMB model, a soft one-sided penalty discourages $\alpha_i < S_{\text{obs},i}$ (the observed species count):

$$\mathcal{R}_\alpha = \frac{1}{|B|} \sum_{i \in B} \left[\frac{\log(1 + e^{\kappa(S_{\text{obs},i} - \alpha_i)})}{\kappa}\right]^2 \tag{6}$$

where $\kappa = 10$ sharpens the hinge boundary and B is the current mini-batch. This is the same softplus-squared penalty used in the TMB model, applied to both α_i and α_j in each pair by indexing into the observed-richness vector.

4 Beta sub-model: monotone turnover network

4.1 TMB approach

In the TMB model, turnover is parameterised using I-spline basis functions with non-negative coefficients:

$$\eta_{i,j} = \exp(\eta_0^{\text{raw}}) + \sum_{l=1}^{m} \sum_{s=1}^{S} \exp(\beta_{l,s}^{\text{raw}}) \cdot |I_s(x_{l,i}) - I_s(x_{l,j})|$$

where $I_s(\cdot)$ are monotonically non-decreasing I-spline bases and the exponentiation of the raw coefficients ensures $\beta_{l,s} \geq 0$. The composition of non-negative coefficients with monotone bases guarantees that $\eta_{i,j}$ is non-decreasing in each $|x_{l,i} - x_{l,j}|$.

4.2 Shared building block: MonotoneLinear layers

All beta network variants rely on a **MonotoneLinear** layer, which is a standard linear layer with the weight matrix constrained to be element-wise non-negative via a softplus transform on unconstrained parameters:

$$\tilde{\mathbf{W}}^{(\ell)} = \text{softplus}(\mathbf{W}_{\text{raw}}^{(\ell)}) \geq 0 \tag{7}$$

The unconstrained $\mathbf{W}_{\text{raw}}^{(\ell)}$ are the learnable parameters; the optimiser updates them freely while the softplus transform guarantees non-negative effective weights. The bias terms $\mathbf{b}^{(\ell)}$ are unconstrained.

Monotonicity proof. Consider the composition at any layer ℓ:

1. The input $\mathbf{x}^{(\ell-1)} \geq \mathbf{0}$ (base case: $\mathbf{\Delta} \geq 0$)
2. The linear map $\tilde{\mathbf{W}}^{(\ell)}\mathbf{x}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ is non-decreasing in $\mathbf{x}^{(\ell-1)}$ because $\tilde{\mathbf{W}}^{(\ell)} \geq 0$
3. ReLU (or softplus) is monotone non-decreasing, so $\sigma(\cdot)$ preserves the ordering
4. By induction, the full composition is monotone non-decreasing in each input dimension

Critically, L_2 **weight decay must not be applied to $\mathbf{W}_{\text{raw}}$**. Standard L_2 regularisation penalises the raw parameters toward zero, but $\text{softplus}(0) = \ln 2 \approx 0.693$ for every weight, which drives all effective weights to the same value and causes *dimensional collapse* — all environmental dimensions receive identical weight regardless of their ecological importance.

4.3 Deep monotone network (`beta_type=``deep''`)

The simplest approach replaces the I-spline transform with a deep monotone MLP:

$$\eta_{i,j} = \mathrm{softplus}(h_\beta(\boldsymbol{\Delta}_{i,j}; \boldsymbol{W}_\beta)) \tag{8}$$

where $\boldsymbol{\Delta}_{i,j} = [|\tilde{x}_{1,i} - \tilde{x}_{1,j}|, \ldots, |\tilde{x}_{m,i} - \tilde{x}_{m,j}|] \geq \mathbf{0}$ is the vector of standardised environmental distances and h_β is a monotone MLP built from Monotone-Linear layers with ReLU activations.

This directly generalises the TMB approach: the I-spline + non-negative coefficient structure *is* a single-layer monotone network with a specific (spline) feature transform. The multi-layer version is a universal approximator for monotone functions (Daniels & Velikova, 2010). However, the deep variant is prone to dimensional collapse (all weights converging under weight decay) and is not the recommended architecture.

4.4 Additive monotone network (`beta_type=``additive''`)

The additive beta network is the direct neural analog of GDM's additive I-spline structure. Turnover decomposes as a sum of independent per-dimension monotone functions:

$$\eta_{i,j} = \sum_{k=1}^{m} f_k(|\tilde{x}_{k,i} - \tilde{x}_{k,j}|) \tag{9}$$

where each $f_k : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is an independent monotone network with n hidden units (default $n = 32$):

$$g_k(d) = \mathbf{w}_2^\top \,\mathrm{softplus}(\tilde{\mathbf{w}}_1\, d + \mathbf{b}_1)\,, \qquad f_k(d) = g_k(d) - g_k(0) \tag{10}$$

Here $\tilde{\mathbf{w}}_1 = \mathrm{softplus}(\mathbf{w}_{1,\mathrm{raw}}) \geq 0$ and $\mathbf{w}_2 = \mathrm{softplus}(\mathbf{w}_{2,\mathrm{raw}}) \geq 0$ are non-negative weight vectors. The hidden activation is **softplus** rather than ReLU: softplus has no dead zone (always nonzero gradient), giving naturally gradual onset of turnover rather than the hard on/off threshold behaviour of ReLU. Softplus is still monotone non-decreasing, preserving the monotonicity guarantee.

The **baseline subtraction** $f_k(d) = g_k(d) - g_k(0)$ ensures that $f_k(0) = 0$ exactly, so zero environmental distance contributes zero turnover. Without this, the network biases would introduce a nonzero η even for identical sites.

The biases $\mathbf{b}_1$ in the first layer play the role of knot locations: they are initialised uniformly from -3σ to 0σ of the standardised environmental distance, spreading the response onset across the observed range. The second-layer weights are initialised very small (at $\mathrm{softplus}(-3) \approx 0.05$ effective weight) so that each dimension initially contributes ≈ 0.3 at maximum distance, giving a total initial $\eta \approx 5$ across all dimensions.

The final output is clamped: $\eta_{i,j} = \mathrm{clamp}(\sum_k f_k, 0, 10)$. The upper bound of 10 corresponds to $S = e^{-10} \approx 4.5 \times 10^{-5}$, effectively complete dissimilarity.

Without this clamp, the model can push $\eta \to \infty$ for all between-site pairs, learning the degenerate solution $p \approx 0$ everywhere.

4.5 Factored deep monotone network (`beta_type=''factored''`)

The factored architecture combines the per-dimension specialisation of the additive network with learned cross-dimension interactions:

$$\eta_{i,j} = \underbrace{\sum_{k=1}^{m} s_k(|\Delta x_k|)}_{\text{additive component}} + \underbrace{\phi(\mathbf{h}_1, \ldots, \mathbf{h}_m)}_{\text{interaction component}} \tag{11}$$

The architecture proceeds in three stages:

1. **Per-dimension monotone encoders**: Each $|\Delta x_k|$ (scalar) is mapped through a private multi-layer monotone encoder to a hidden representation $\mathbf{h}_k \in \mathbb{R}_{\geq 0}^{d_{\text{enc}}}$ (default $d_{\text{enc}} = 16$, depth 2). The final activation is ReLU, ensuring $\mathbf{h}_k \geq 0$ element-wise.

2. **Per-dimension scalar projections**: Each $\mathbf{h}_k$ is projected to a scalar $s_k = \mathbf{w}_k^\top \mathbf{h}_k$ via a MonotoneLinear$(d_{\text{enc}}, 1)$ layer, then baseline-subtracted: $s_k(\Delta x_k) - s_k(0)$. The sum $\sum_k s_k$ forms the additive component.

3. **Monotone interaction network**: The concatenation $[\mathbf{h}_1; \ldots; \mathbf{h}_m] \in \mathbb{R}^{m \cdot d_{\text{enc}}}$ is fed through a shared monotone MLP (MonotoneLinear layers with softplus activation) producing a scalar interaction term. This is also baseline-subtracted: $\phi(\mathbf{h}) - \phi(\mathbf{0})$.

Monotonicity guarantee: Each encoder is monotone in its input (non-negative weights + monotone activations). The interaction network has non-negative weights (MonotoneLinear) and takes non-negative inputs ($\mathbf{h}_k \geq 0$ via ReLU), so it is monotone in each $\mathbf{h}_k$. Since the composition of monotone functions is monotone, the full network is monotone in each $|\Delta x_k|$. Dropout is disabled throughout the factored network because baseline subtraction requires deterministic forward passes.

The interaction term allows the model to capture effects such as: the impact of temperature distance on turnover depends on the magnitude of precipitation distance. The additive network (Section 4.4) cannot represent such interactions.

4.6 Geographic distance as a beta input

Optionally, the beta network can include geographic (Haversine) distance as an additional input dimension:

$$\mathbf{\Delta}_{i,j} = \left[|\tilde{x}_{1,i} - \tilde{x}_{1,j}|, \ldots, |\tilde{x}_{m,i} - \tilde{x}_{m,j}|, \ \frac{d_{\text{geo}}(i,j)}{\sigma_{\text{geo}}} \right] \tag{12}$$

where $d_{\text{geo}}(i, j)$ is the great-circle distance in kilometres and σ_{geo} is estimated from random site pairs. This additional dimension gets its own monotone sub-network (additive or encoded) and allows the model to capture *distance decay* — the empirical observation that community similarity decreases with geographic distance independent of environmental differences (e.g. dispersal limitation).

The geographic distance feature is typically introduced in Phase 2 of the cyclic fine-tuning procedure (Section 5.2.4), after the environmental signal has been learned.

4.7 Comparison

	TMB (I-spline)	Additive NN	Factored NN
Monotonicity	Non-neg coef $\times$ I-spline	Non-neg weights $\times$ softplus	Non-neg weights $\times$ softplus
Flexibility	$S \times m$ params	$n \times m$ params	$n \times m$ + interaction
Interactions	None (additive)	None (additive)	Learned via shared network
$f_k(0) = 0$	By construction	Baseline subtraction	Baseline subtraction
η range	$[0, \infty)$	$[0, 10]$ clamped	$[0, 10]$ clamped
Collapse risk	Low	Low	Low (per-dim encoders)

The additive network is the default (`beta_type=''additive''`) as it most closely mirrors the GDM/TMB structure and produces directly interpretable per-variable response curves. The factored variant is available when cross-variable interactions are suspected.

5 Optimisation

The TMB model is fitted via maximum marginal likelihood using `nlminb`, with the Laplace approximation integrating over the random effects $\mathbf{u}$. Each function evaluation requires a sparse Cholesky factorisation of the joint precision matrix, which is $\mathcal{O}(n)$ in the number of sites but with a large constant.

The neural network is fitted via mini-batch stochastic gradient descent using Adam (Kingma & Ba, 2015). Each parameter update costs $\mathcal{O}(B \cdot D)$ where B is the batch size and D is the total number of network parameters. There is no Hessian or Laplace approximation.

	TMB	Neural network
Optimiser	`nlminb` (quasi-Newton)	Adam (adaptive SGD)
Random effects	Laplace approximation	None (absorbed by network)
Per-iteration cost	Sparse Cholesky ($\sim$ seconds)	Matrix multiplies ($\sim$ milliseconds)
Typical runtime	Hours to days	Minutes
Hardware	CPU only	CPU, GPU, or Apple MPS

Training details:

- **Learning rate scheduling**: Cosine annealing with warm restarts ($T_0 =$ 30 epochs, $T_{\text{mult}} = 2$), giving cycle lengths of 30, 60, 120,... epochs. The minimum LR is 10^{-6}.
- **Early stopping**: halt training if validation loss does not improve for a configurable number of epochs (default 40).
- **Gradient clipping**: max norm of 5.0 on all parameters to prevent exploding gradients.
- **Weight decay**: L_2 penalty of 10^{-5} on alpha parameters and beta bias parameters. *No weight decay* on beta `weight_raw` parameters (see Section 4.4).
- **Dropout**: applied between hidden layers (default rate 0.1) to reduce overfitting.
- **Batch size**: 8192 pairs per mini-batch (default).

5.1 Gradient imbalance between alpha and beta

A fundamental challenge in jointly training the alpha and beta sub-models is a severe gradient imbalance. The match probability for between-site pairs is:

$$p_{\text{between}} = \underbrace{e^{-\eta}}_{S} \cdot \frac{\alpha_i + \alpha_j}{2\,\alpha_i\,\alpha_j}$$

Differentiating the log-likelihood with respect to η introduces a factor of $\partial p/\partial \eta = -p$, so beta gradients are attenuated by a factor of approximately $p \sim 1/\alpha$. For realistic richness values ($\alpha \approx 3$–100), beta gradients can be 100–1000× smaller than alpha gradients, effectively starving the turnover network of learning signal.

Three mechanisms address this imbalance:

1. **Beta LR multiplier**: The beta network receives a higher base learning rate than alpha (default 10×), implemented via separate Adam parameter groups.
2. **Gradient amplification hooks**: PyTorch backward hooks on beta parameters scale their gradients by a configurable factor (default 100× in joint training, 10× in two-stage/cyclic modes). This is applied *before* gradient clipping.
3. **Importance weighting** (in Stage 2 / beta phases): Match pairs ($y = 0$) are reweighted by $1/\alpha_h$ where $\alpha_h = 2\alpha_i\alpha_j/(\alpha_i+\alpha_j)$ is the harmonic mean richness. This balances the gradient contributions of match vs. mismatch pairs:

$$\mathcal{L}_{\text{beta}} = \frac{\sum_p \tilde{w}_p \cdot \text{BCE}(y_p, p_p)}{\sum_p \tilde{w}_p}, \qquad \tilde{w}_p = \begin{cases} 1/\alpha_h & \text{if } y_p = 0 \text{ (match)} \\ 1 & \text{if } y_p = 1 \text{ (mismatch)} \end{cases} \tag{13}$$

5.2 Training modes

Four training modes are implemented, addressing the alpha–beta gradient imbalance with increasing sophistication:

5.2.1 Joint training (`mode=''joint''`)

All parameters are optimised simultaneously on all pairs. This is the simplest mode but is prone to *beta collapse*: alpha inflates to match sampled pair frequencies rather than true richness, making $\partial p/\partial \eta \approx 0$ and leaving beta unable to learn. Gradient amplification and the beta LR multiplier partially mitigate this, but joint training is not recommended for production use.

5.2.2 Two-stage training (`mode=''two_stage''`)

Mirrors the original CLESSO two-stage approach (`clesso_alpha.cpp` → `clesso_beta_fixAlpha.cpp`):

Stage 1: Freeze beta, train alpha on *within-site pairs only* with BCE + lower-bound penalty. Alpha converges to realistic richness values ($\alpha \approx S_{\text{obs}}$).

Stage 2: Freeze alpha, train beta on *between-site pairs only* with importance-weighted BCE. Beta receives uncontested, balanced gradients and learns non-trivial turnover response curves.

Each stage uses its own optimizer (Adam with cosine annealing) and early stopping.

5.2.3 Cyclic block-coordinate descent (`mode=''cyclic''`)

Mirrors `clesso_iterative.R`: alternating block-coordinate optimisation with damped step sizes:

for cycle $= 1, \ldots, C_{\max}$: (A) freeze β, train α for N_α epochs (B) freeze α, train β for N_β epochs

Default settings use $N_\alpha = 5$ and $N_\beta = 10$ (small alpha steps, 2× beta steps), keeping alpha changes incremental so beta can adapt without oscillation. Convergence is declared when the relative change in best beta validation loss between cycles falls below a threshold (10^{-4}).

Key differences from two-stage:

- Alpha and beta *see each other's updated parameters repeatedly*, allowing co-adaptation.
- Alpha can re-adjust after beta has learnt turnover (e.g. correcting inflated estimates).
- Persistent optimiser state (momentum) is maintained across cycles.
- Step-based LR decay (halve every 200 global epochs) replaces per-stage cosine annealing.

Cyclic training is the default mode (`training_mode=''cyclic''`).

5.2.4 Cyclic + geographic fine-tuning (`mode=''cyclic_finetune''`)

A two-phase procedure that separates environmental and geographic signal learning:

Phase 1: Damped cyclic training on *environmental and substrate covariates only* (no Fourier features, no geographic distance). This learns the core ecological signal without geographic confounding.

Phase 2: Expand the model architecture to accommodate new input dimensions:

- Alpha: add Fourier positional encoding features (zero-initialised weights so Phase 1 predictions are initially unchanged).
- Beta: add a geographic distance dimension (new per-dimension monotone sub-network with fresh random initialisation).

Fine-tune with a smaller learning rate. Optionally, Phase 1 parameters can be frozen (`finetune_freeze_existing=True`) so only the new geographic parameters are trained.

This mirrors the intuition from spatial ecology that environmental variables should explain most compositional turnover, with geographic distance capturing residual dispersal limitation.

5.3 Hard-pair mining with importance correction

The pairwise likelihood in equation (2) is defined with respect to a *target pair distribution* over the sampled pair set P, represented by the existing cell-balancing weights w_p. Naïve hard-pair mining changes this training distribution by over-sampling difficult pairs, which can improve optimisation speed but distort the resulting fitted probabilities. To retain the original probabilistic target while focusing optimisation on informative pairs, hard-pair mining is introduced as a *proposal distribution* with importance correction.

Target and proposal distributions. Let $\pi(p)$ denote the target sampling probability for pair p, corresponding to the original pair-construction design, and let $q(p)$ denote the actual probability that pair p is selected into a mini-batch under the mining scheme. The training loss uses an importance-corrected weight:

$$w_p^\star = w_p \cdot \frac{\pi(p)}{q(p)} \tag{14}$$

In practice, three layers of importance correction are composed multiplicatively. For a between-site pair in a mined mini-batch, the effective weight is:

$$w_p^{\text{eff}} = \underbrace{w_p}_{\substack{\text{population}\\ \text{proportion}}} \cdot \underbrace{\frac{1}{\alpha_h}}_{\substack{\text{gradient}\\ \text{balance}}} \cdot \underbrace{\frac{\pi(p)}{q(p)}}_{\substack{\text{mining}\\ \text{correction}}} \tag{15}$$

where $\alpha_h = 2\alpha_i\alpha_j/(\alpha_i+\alpha_j)$ is the harmonic mean richness from Section 5.1. The first factor corrects for the 50/50 match/mismatch sampling design, the second balances the gradient attenuation from Section 5.1, and the third corrects for the non-uniform mini-batch sampling introduced by mining. When $q(p) = \pi(p)$, the mining correction is unity and the loss reduces to the original importance-weighted likelihood.

The mini-batch loss is:

$$\widehat{\mathcal{L}}_B = -\frac{1}{\sum_{p\in B} w_p^{\text{eff}}} \sum_{p\in B} w_p^{\text{eff}} \Big[(1-y_p)\log p_p + y_p \log(1-p_p)\Big] + \lambda_{\text{lb}} \cdot \mathcal{R}_\alpha \quad (16)$$

Coarse pair strata. Hard-pair mining is applied only to *between-site* pairs during beta training phases, since within-site pairs anchor the alpha model and have a fundamentally different loss surface ($p = 1/\alpha$, no η dependence). Between-site pairs are divided into two coarse cells:

$$c(p) \in \{\text{between-match, between-mismatch}\}$$

Each cell receives equal target mass, so the target distribution is:

$$\pi(p) = \Pr(c(p)) \cdot \frac{1}{N_{c(p)}} = \frac{1}{2} \cdot \frac{1}{N_{c(p)}} \quad (17)$$

where N_c is the number of training pairs in cell c.

Difficulty strata. For each between-site pair p, a hardness score is computed using the current per-pair negative log-likelihood:

$$h_p = -\Big[(1-y_p)\log p_p + y_p \log(1-p_p)\Big] \quad (18)$$

Within each coarse cell, pairs are ranked by h_p and partitioned into D difficulty bins (default $D = 3$: easy, medium, hard) using quantile boundaries. Let $d(p)$ denote the difficulty bin for pair p, and $N_{c,d}$ the number of pairs in cell c and bin d.

Using quantile-based bins rather than absolute NLL thresholds avoids a subtle issue: pairs near the $\eta = 10$ clamp boundary (Section 4.4) have irreducibly high NLL due to the modelling constraint, not because they are informative. Quantile ranking within each cell ensures these pairs are distributed across bins rather than dominating the "hard" bin.

Mixture proposal. The proposal distribution for between-site pairs is a mixture:

$$q(p) = (1-\lambda_{\text{hm}})\,\pi(p) + \lambda_{\text{hm}}\, h^\star(p) \quad (19)$$

where $\lambda_{\text{hm}} \in [0,1]$ controls the mining strength and $h^\star(p)$ is the hard-pair proposal:

$$h^\star(p) = \Pr(c(p)) \cdot \Pr(d(p) \mid c(p)) \cdot \frac{1}{N_{c(p),d(p)}} \quad (20)$$

The conditional bin probabilities $\Pr(d \mid c)$ assign extra mass to difficult bins (default: $[0.2, 0.3, 0.5]$ for easy/medium/hard). At $\lambda_{\text{hm}} = 0$ (default), the proposal equals the target and all mining code is a no-op.

Variance control. The importance ratio $\pi(p)/q(p)$ can become large when easy pairs are drawn from the hard-biased proposal. To prevent unstable gradients, the ratio is truncated:

$$\frac{\pi(p)}{q(p)} \leftarrow \min\left(\frac{\pi(p)}{q(p)},\ a_{\max}\right) \tag{21}$$

with $a_{\max} = 5$ by default. This introduces a small bias in exchange for substantially reduced gradient variance. The truncation interacts with the existing $1/\alpha_h$ importance weighting: high-α match pairs from easy bins receive the product of both corrections, so the truncation cap prevents doubly extreme weights.

Practical training scheme. Training proceeds in two phases:

1. **Warm-up.** For the first few cycles (default 3), mini-batches are sampled from the target distribution only ($\lambda_{\text{hm}} = 0$). This avoids using unstable model-based hardness scores early in training when the beta network has not yet learned meaningful turnover.

2. **Mining phase.** At the start of each subsequent beta training phase (Phase B of cyclic training, or each epoch in two-stage mode), a forward pass over all between-site training pairs computes hardness scores h_p, refreshes the difficulty bins, and rebuilds the proposal distribution $q(p)$. Scores can optionally be refreshed mid-phase (default: every beta epoch) to track the evolving loss landscape.

The scoring forward pass is inexpensive — it amounts to one epoch of inference over the training pairs with no gradient computation — and is easily amortised over the subsequent training epochs.

Interpretation. This scheme preserves the original probabilistic target of the weighted Bernoulli likelihood while allowing optimisation to focus more heavily on informative between-site pairs. In effect, hard-pair mining is treated as a *sampling design for stochastic optimisation* rather than a change to the statistical model itself. The fitted match probabilities remain calibrated with respect to the original pair distribution, which is critical because downstream predictions (beta-diversity indices, compositional similarity maps) rely on these probabilities being interpretable rather than merely ordinal.

6 Relationship to TMB model

The neural network variant is not a different model in the ecological sense. The likelihood (equation 2), the match probability decomposition (equation 1), and

the pair sampling strategy are all identical. What changes is the function class used to map covariates to α and environmental distances to η:

Component	TMB	Neural network
α_i	Linear + P-spline + random effect	MLP (SiLU) with softplus output
$\eta_{i,j}$	I-spline + non-negative coefficients	Additive/factored monotone network
p_p	Same formula	Same formula
$\mathcal{L}$	Same weighted Bernoulli NLL	Same (+ importance weighting for β)
$\mathcal{R}_\alpha$	Same softplus^2 lower-bound penalty	Same softplus^2 lower-bound penalty
Training	`nlminb`, two-stage, iterative	Adam, two-stage/cyclic block-coordinate
Geography	I-spline on geo distance	Fourier encoding (α) + haversine dim (β)

The neural network can be viewed as replacing the manually specified basis functions (B-splines, I-splines) with learned feature representations, and replacing the Gaussian random effects with implicit regularisation through dropout and weight decay. The additive beta network (default) preserves the per-variable interpretability of GDM while gaining the flexibility of learned (rather than pre-specified) knot placements.

7 Post-training: diagnostics and spatial prediction

After training, the pipeline automatically runs three post-processing steps:

7.1 Diagnostics

A comprehensive diagnostic PDF (mirroring the R script `clesso_diagnostics.R`) is generated, containing:

- Training curves (loss, accuracy, alpha statistics, learning rate schedule)
- Residual analysis (within-site and between-site)
- Observed vs. predicted match probability
- Classification metrics (AUC, confusion matrix)
- Spatial map of fitted α (richness)
- α vs. observed species richness comparison
- Beta turnover response curves per environmental dimension (analogous to I-spline partial functions)
- Similarity decay curves $S = \exp(-\eta)$
- Alpha partial-dependence plots (site covariates $\rightarrow$ richness)
- Variable importance bar chart

7.2 Alpha surface

Species richness is predicted across all of Australia using the trained alpha network applied to a gridded covariate stack (0.05° resolution, 670×813 cells). The covariates include 6 substrate PCA bands, 9 AWAP climate variables (matching training columns), and optionally Fourier positional features. The output is a single-band GeoTIFF.

7.3 Beta surface (community composition map)

A spatial map of biological community composition is generated using Landmark MDS:

1. Select k landmark pixels via spatial k-means (default $k = 500$)
2. Compute pairwise η between all landmark pairs using the trained beta network ($k \times k$ matrix)
3. Compute η from every pixel to each landmark ($n \times k$ matrix)
4. Convert to dissimilarity: $D = 1 - \exp(-\eta)$
5. Classical MDS on the landmark dissimilarity matrix + Nyström extension for all pixels
6. Map 3 MDS dimensions to RGB channels

The result is a 3-band GeoTIFF where similar colours indicate similar predicted community composition.

8 Further thoughts

- All the interpretive considerations from the TMB document apply: α represents "effective richness under the recording process" rather than true species counts, and the equal-draw assumption means our estimates conflate richness with evenness.
- The additive monotone network can be interrogated post-hoc by sweeping each environmental dimension while holding others at zero, producing per-variable response curves analogous to the I-spline partial functions in GDM. The factored network additionally allows examination of joint response surfaces.
- The ability to capture variable interactions (factored beta) is a double-edged sword — more flexible, but harder to interpret. Careful examination of joint response surfaces is recommended. The additive beta is preferred when interpretability is paramount.

- The network's richness predictions should be compared against the TMB P-spline predictions on the same data to assess consistency between the two approaches.
- If uncertainty quantification is needed (which the TMB `sdreport` provides), options include MC dropout at prediction time, deep ensembles, or training multiple models with different random seeds.
- The data pipeline (Steps 1–4 of the R pipeline) is shared between both approaches, so switching between TMB and NN fitting requires only an export step (`export_for_nn.R`) and a single Python command.
- The cyclic block-coordinate descent training (default mode) most closely mirrors the iterative R pipeline (`clesso_iterative.R`), alternating alpha and beta optimisation until convergence.
- The two-phase cyclic fine-tuning procedure (`cyclic_finetune`) provides a principled way to separate environmental signal from geographic distance decay, reducing the risk of geographic confounding during initial model fitting.
- The observed-richness lower-bound penalty should use *corrected* S_{obs} values (true species counts from raw data) rather than the subsampled pair-source richness, which underestimates true counts.